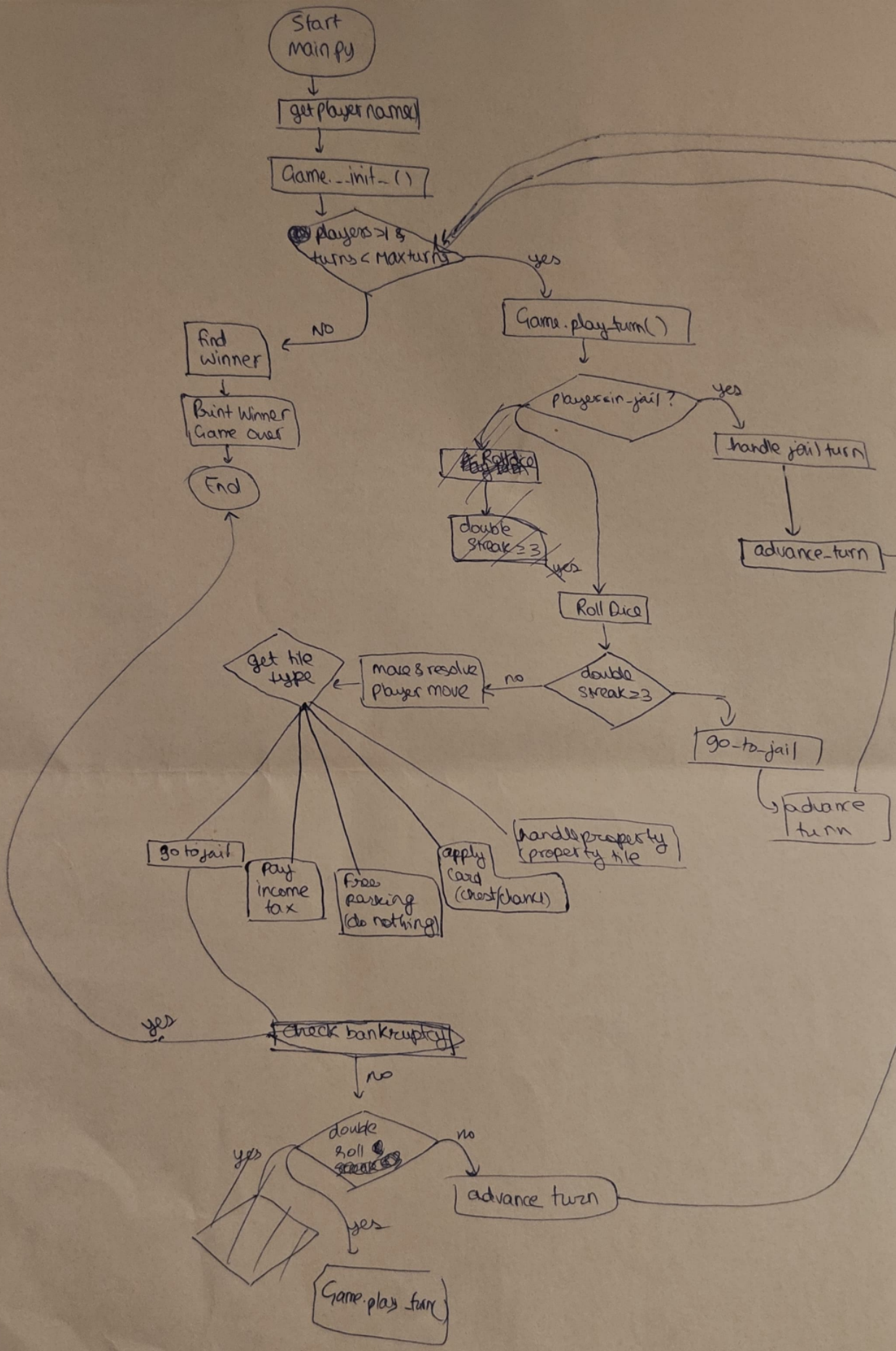
Start main.py
get player names()
Game.__init__()
players > 1 & turns < Max turns
yes
NO
find winner
Print Winner Game over
End
Game.play_turn()
player in_jail ?
yes
handle jail turn
advance_turn
double streak ≥ 3
yes
Roll Dice
double streak ≥ 3
no
move & resolve player move
get tile type
go_to_jail
advance turn
go to jail
Pay income tax
Free parking (do nothing)
apply card (chest/chance)
handle property property tile
yes
check bankruptcy
no
double roll
yes
no
advance turn
yes
Game.play_turn

Part - 1 Report

Part 1.2

Fixes

Iteration 1: Fixed import errors by adding a `__init__.py` to the `moneypoly` directory.
Iteration 2: added docstrings and removed unused import from `moneypoly/bank.py`.
Iteration 3: added module docstring and changed '==' based comparison to simply 'is' comparison in `moneypoly/board.py`.
Iteration 4: added module docstrings and reformatted long lines in `moneypoly/cards.py`.
Iteration 5: added module docstring in `moneypoly/config.py`.
Iteration 6: removed unused import of `BOARD_SIZE`, added module docstring, defined `double_streak` attribute in `__init__` for Dice class in `moneypoly/dice.py`.
Iteration 7: added module docstring, removed unused imports (`os`, `GO_TO_JAIL_POSITION`), fixed unnecessary `elif` after `break`, and added missing final newline in `moneypoly/game.py`.
Iteration 8: removed unnecessary parens after 'not' keyword, used normal print statement instead of f-string since no interpolated variables in `moneypoly/game.py`.
Iteration 9: added newline at last line, module docstring, removed unused `sys` import, removed unused variable `old_position` in `moneypoly/player.py`.
Iteration 10: added module docstring and class docstring for `PropertyGroup`, removed unnecessary `else` after `return` in `unmortgage` in `moneypoly/property.py`.
Iteration 11: added module docstring and changed bare `except` to `except ValueError` in `safe_int_input` in `moneypoly/ui.py`.
Iteration 12: changed the `is_eliminated` property to be a class attribute (used to be instance attribute) since it is an immutable data type in `moneypoly/player.py`.
Iteration 13: changed `turn_number`, `running`, `current_index`, `board` to be class attributes instead of instance attributes in the `Game` class since there's only ever 1 instance of `Game`, refactored `_apply_card` to fix the too-many-branches warning by using dispatch dictionary by defining helper functions in `moneypoly/game.py`.
Iteration 14: changed `is_mortgaged`, `houses` to be a class property from an instance property, changed `mortgage_value` to a property method in the `Property` class, also changed `group` to be read as a keyword argument that is to be passed whenever property is made in `moneypoly/property.py`. Changed `moneypoly/board.py` to accomodate this.
Iteration 15: added function and module docstrings to `main.py`.

Part 1.3

Test Descriptions

test_main.py

1. `test_single_player` - Creating a game with 1 player should raise an error [**FAIL**]
2. `test_zero_players` - Creating a game with 0 players should raise an error [**FAIL**]
3. `test_duplicate_names` - Creating a game with duplicate names should raise an error [**FAIL**]

test_player.py

1. `test_add_money_negative` - add_money(-1) raises ValueError
2. `test_add_money_positive` - add_money(100) increases balance by 100
3. `test_deduct_money_negative` - deduct_money(-1) raises ValueError
4. `test_deduct_money_beyond_balance` - deduct_money(2000) on balance 1500 results in -500
5. `test_is_bankrupt` - balance 0 is bankrupt, balance 1 is not
6. `test_move_land_on_go` - Moving from pos 38 by 2 lands on Go and awards salary
7. `test_move_pass_go` - Moving from pos 38 by 4 passes Go but salary is not awarded [**FAIL**]
8. `test_add_property_twice` - Adding the same property twice results in only one entry
9. `test_remove_property_not_owned` - Removing a property not owned causes no error
10. `test_net_worth_with_property` - net_worth does not include property values [**FAIL**]

test_property.py

1. `test_get_rent_mortgaged` - Rent on a mortgaged property is 0
2. `test_get_rent_partial_ownership` - Owning 1 of 2 group properties returns doubled rent instead of base rent [**FAIL**]
3. `test_get_rent_no_group` - Property with no group returns base rent
4. `test_mortgage_already_mortgaged` - Mortgaging an already mortgaged property returns 0
5. `test_mortgage_unmortgaged` - Mortgaging sets flag and returns half price
6. `test_unmortgage_not_mortgaged` - Unmortgaging a non-mortgaged property returns 0
7. `test_unmortgage_mortgaged` - Unmortgaging clears flag and returns cost with 10% interest
8. `test_is_available` - is_available checks unowned and not mortgaged
9. `test_all_owned_by_none` - all_owned_by(None) returns False

10. `test_all_owned_by_partial` - Owning 1 of 2 properties incorrectly returns True (uses any instead of all) **[FAIL]**

test_bank.py

1. `test_collect_negative` - collect(-100) should not change bank funds **[FAIL]**
2. `test_pay_out_zero` - pay_out(0) returns 0, funds unchanged
3. `test_pay_out_insufficient` - pay_out more than funds raises ValueError
4. `test_pay_out_valid` - pay_out(100) deducts from funds and returns 100
5. `test_give_loan_zero` - give_loan with amount 0 does nothing
6. `test_give_loan_valid` - give_loan does not deduct from bank funds **[FAIL]**
7. `test_give_loan_insufficient_funds` - give_loan with insufficient bank funds is not rejected **[FAIL]**

test_dice.py

1. `test_doubles_streak_increment` - Rolling doubles increments streak, non-doubles resets it

test_cards.py

1. `test_draw_empty` - draw() on empty deck returns None
2. `test_draw_wraps` - draw() past end wraps back to first card
3. `test_cards_remaining` - cards_remaining decrements after a draw
4. `test_shuffle_on_init` - Deck is not shuffled on initialization **[FAIL]**
5. `test_reshuffle_after_exhaustion` - Deck is not reshuffled after all cards are drawn **[FAIL]**

test_board.py

1. `test_get_property_at` - Returns correct property or None for non-property positions
2. `test_get_tile_type_special` - Special tiles return correct type strings
3. `test_get_tile_type_property` - Property position returns "property"
4. `test_get_tile_type_blank` - Unmodelled position returns "blank"
5. `test_is_purchasable_and_is_special` - Unowned property is purchasable, owned is not, Go is special

test_config.py

1. `test_special_tiles_unique` - All special tile positions are unique
2. `test_no_property_in_special` - No property overlaps with a special tile
3. `test_positions_within_bounds` - All positions are within board bounds
4. `test_property_positions_distinct` - All property positions are distinct

5. `test_card_move_to_values` - All card move_to values are within board bounds

test_game.py

1. `test_advance_turn_empty` - advance_turn on empty player list crashes with ZeroDivisionError [**FAIL**]
2. `test_play_turn_in_jail` - Jailed player calls _handle_jail_turn
3. `test_play_turn_three_doubles_jail` - 3 consecutive doubles sends player to jail
4. `test_play_turn_doubles_extra_turn` - Doubles with streak < 3 does not advance turn
5. `test_play_turn_normal` - Non-doubles roll advances turn
6. `test_interactive_menu` - play_turn does not call interactive_menu before rolling [**FAIL**]
7. `test_doubles_streak_bleeds` - Doubles streak is not reset between different players' turns [**FAIL**]
8. `test_move_and_resolve_all_tiles` - Each tile type triggers the correct handler
9. `test_buy_property_exact_balance` - Buying with balance equal to price fails instead of succeeding [**FAIL**]
10. `test_buy_property_normal` - Balance below price fails, above succeeds
11. `test_pay_rent_mortgaged` - Mortgaged property charges no rent
12. `test_pay_rent_no_owner` - Unowned property charges no rent
13. `test_pay_rent_normal` - Rent is not transferred from tenant to owner [**FAIL**]
14. `test_mortgage_property` - Not owner, already mortgaged, and success cases
15. `test_unmortgage_property` - Not owner, not mortgaged, and success cases
16. `test_unmortgage_cant_afford` - Unmortgage clears mortgage flag before checking balance [**FAIL**]
17. `test_trade` - Seller not owning and buyer too poor cases, plus successful trade [**FAIL**]
18. `test_auction_all_pass` - All players bid 0 means property stays unowned
19. `test_auction_bid_below_min` - Bid below minimum increment is rejected
20. `test_auction_bid_above_balance` - Bid above balance is rejected
21. `test_auction_valid_bid` - Valid bid awards property and charges winner
22. `test_jail_free_card` - Using get-out-of-jail card releases player and consumes card
23. `test_jail_pay_fine` - Paying fine does not deduct JAIL_FINE from balance [**FAIL**]
24. `test_jail_decline_under_3` - Declining fine with turns < 3 keeps player jailed

25. `test_jail_mandatory_release` - Forced release at turn 3 deducts fine
26. `test_jail_fine_bankruptcy` - Mandatory jail fine bankrupting player does not eliminate them [**FAIL**]
27. `test_card_move_to_backward_salary` - Moving backward past Go awards salary
28. `test_card_move_to_forward_no_salary` - Moving forward without passing Go gives no salary
29. `test_card_move_to_railroad` - Railroad destination does not trigger property handling [**FAIL**]
30. `test_card_birthday` - Birthday card deducts from rich players, skips poor ones
31. `test_apply_card_none` - _apply_card with None returns immediately
32. `test_apply_card_jail` - Jail card sends player to jail
33. `test_apply_card_collect` - Collect card adds value to balance
34. `test_apply_card_unknown` - Unknown card action is silently ignored
35. `test_check_bankruptcy_not_bankrupt` - Non-bankrupt player stays in the game
36. `test_check_bankruptcy_bankrupt` - Bankrupt player is eliminated and properties released
37. `test_check_bankruptcy_last_in_list` - Bankrupting last player resets current_index to 0
38. `test_trade_bankruptcy` - Trade bankrupting buyer does not eliminate them [**FAIL**]
39. `test_auction_bankruptcy` - Auction bankrupting winner does not eliminate them [**FAIL**]
40. `test_birthday_bankruptcy` - Birthday card bankrupting player does not eliminate them [**FAIL**]
41. `test_find_winner_no_players` - No players returns None
42. `test_find_winner_richest` - find_winner does not return the richest player [**FAIL**]

Summary

- **Passed:** 62
- **Failed:** 26
- **Total:** 88

Fixes

Error 1: Added early return in `Bank.collect()` if amount negative.
Error 2: Added `self._funds -= amount` in `Bank.give_loan()` to deduct from bank funds.
Error 3: Added insufficient funds check in `Bank.give_loan()` raising ValueError.
Error 4: Added `random.shuffle(self.cards)` in `CardDeck.__init__()` to shuffle on initialization; added `shuffle=False` parameter and used it in

ordered-list tests.
Error 5: Added reshuffle in `CardDeck.draw()` when the deck wraps aroud, only done when`self._shuffle` flag is set.
Error 6: Added early return in `Game.advance_turn()` if players list is empty.
Error 7: Added `self.interactive_menu(player)` call in `Game.play_turn()` before rolling; updated affected tests to mock `interactive_menu`.
Error 8: Added `self.dice.doubles_streak = 0` in `Game.advance_turn()` to reset streak between players.
Error 9: Changed `<=` to `<` in `Game.buy_property()` so equal balance is sufficient to buy.
Error 10: Added `prop.owner.add_money(rent)` in `Game.pay_rent()` to transfer rent to owner.
Error 11: Moved balance check before `prop.unmortgage()` in `Game.unmortgage_property()` to prevent clearing mortgage when player can't afford it.
Error 12: Added `seller.add_money(cash_amount)` in `Game.trade()` to transfer cash to seller.
Error 13: Added `player.deduct_money(JAIL_FINE)` in `Game._handle_jail_turn()` when player voluntarily pays fine.
Error 14: Added `_check_bankruptcy()` call after mandatory jail fine in `Game._handle_jail_turn()`.
Error 15: Added `"railroad"` to tile check in `Game._card_move_to()` so railroad destinations trigger property handling.
Error 16: Added `_check_bankruptcy(buyer)` in `Game.trade()` after buyer pays.
Error 17: Added `_check_bankruptcy(highest_bidder)` in `Game.auction_property()` after winner pays.
Error 18: Added `_check_bankruptcy(other)` in `Game._card_birthday()` after deducting from each player.
Error 19: Changed `min()` to `max()` in `Game.find_winner()` so that player with max net worth wins.
Error 20: Added `ValueError` raise to `Game.__init__()` if less than 2 players are passed.
Error 21: Added `ValueError` raise to `Game.__init__()` to enforce unique player names.
Error 22: Added `self.position <= old_position or steps >= BOARD_SIZE` as conditions to add `GO_SALARY` to player's balance in `Player.move()`.
Error 23: Added sum of each properties price to net worth in `Player.net_worth()`.
Error 24: Changed `any` to `all` in `Property.all_owned_by()` to enforce necessity of complete ownership of all properties in the group.